Algorithm and Learning for Protein Science : From local explanations to global understanding with explainable AI for trees

Guillaume Vindry
guillaume.vindry@gmail.com
ENS Paris-Saclay

Corentin Pernot
corentin.pernot@ensae.fr
ENS Paris-Saclay

1 Introduction

We chose to deep dive into the paper "From local explanations to global understanding with explainable AI for trees" by Lundberg et al. in order to better understand the behavior of complex tree-based machine learning models. This work introduces TreeExplainer, an algorithm that allows for fast and exact computation of SHAP values by leveraging the tree structure. The authors build upon principles from cooperative game theory to derive local feature attributions that are consistent, accurate and also able to capture feature interactions. Beyond local explanations, the paper proposes methods to aggregate these into global insights so as to highlight meaningful patterns across the entire dataset. In this report, we aim to underline the main contributions of the paper, explore the methodology, and also discuss its applications, particularly in the medical domain with the mortality dataset. We also try to reproduce some results and we did some experiments. The code used for our experiments is available at: `https://github.com/CorentinPernot/TreeShap`

1.1 Context and Motivation

Tree models (Random Forests, Gradient Boosting) are widely used nowadays in various domains (insurance, biology, economics, ...). However, their high performance in predicting particular values does not mean that we understand how these predictions are made. We would like to be able to justify a prediction, based on the input data, thus interpretability is crucial : Biomedical applications in particular require transparency to be recognized and accepted in practical applications.

Tree-based models have the advantage over other machine learning methods of being composed of simple elementary bricks : decision trees, which are easily interpretable. However, once the number and the complexity of these trees increases, it is no longer possible for a human to understand the predictions. Thus, there is a need for a method that balances accuracy, interpretability, and speed.

2 Existing methods for tree models and their limitations

Before the introduction of this paper, several methods had already been proposed to derive interpretable insights from tree-based models. Here, we will present some of these existing approaches and highlight their main limitations, which motivated the need for a new solution.

2.1 Decision path reporting

This method is very effective for a single tree. It consists in checking the decision path used to compute the prediction. However, for ensemble models, there are too many trees and their interactions can be complex.

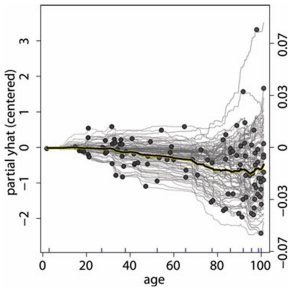

Figure 1: Example of an insightful use of ICE on housing price prediction, from (5) : interactions between the age and other features can lead to a difference of up to 14%. Highlighted : the average CP plot.

2.2 Model-agnostic methods

Here we present a few methods for black-box models interpretability, following this introductory book : (3). These methods try to understand what the model does by feeding it some data points and looking at the output. As highlighted in the Lundberg paper, they can be computationally expensive, especially for large datasets or models with many features, since each explanation requires numerous forward passes through the model. Moreover, their reliance on sampling introduces estimation variance, which can result in unstable explanations. These methods can be applied to any type of model and therefore they do not leverage the structure of the trees.

2.2.1 CP plot (4)

Plot the variation of the output when changing only one feature. Easy to do, but the interaction between the features are not represented.

2.2.2 Individual Conditional Expectation (ICE) (5)

Plotting all the CP plots of the points of the dataset allows us to notice if the variability is conditional on other features, see Fig. 1. However, ICE and CP only show us what happens around the point, not what features determined the prediction at that exact point.

2.2.3 Local Interpretable Model-agnostic Explanations (LIME) (6)

LIME consists in fitting a simple model (typically linear) to a synthetic dataset generated by perturbations around the datapoint to explain. This creates local linear coefficients on a subset of features, which is very useful for interpretability. However, this method is computationally expensive.

2.3 Heuristic methods for tree ensembles : Saabas' method

Unlike model-agnostic approaches, heuristic methods such as Saabas' method are specifically designed to leverage the structure of tree-based models. This method works by computing local feature attributions by tracing the decision path taken by a single input through each tree in the ensemble. At each node, the method attributes the change in the model's expected output to the feature used for the split. The total contribution of a feature is accumulated as the difference in the prediction between parent and child nodes.

Using this notation :

- f: The decision function of the model.
- x: The input instance for which we compute feature attributions.
- $f_x(S) \approx \mathbf{E}[f(x)|x_S]$: The expected output of the model given feature subset S.

The attribution for feature i in Saabas' method is computed as:

$$\phi_i^s(f,x) = \sum_{j \in D_x^i} \big(f_x(A_j \cup \{j\}) - f_x(A_j)\big)$$

where D_x^i is the set of nodes along the decision path that split on feature i, and A_j represents features split on by ancestors of node j. However, this is not justified mathematically and has not been published in a scientific paper.

Although this approach is computationally efficient, the attributions produces are biased: features closer to the root of the tree tend to receive less credit than those closer to the leaves.

2.4 Summary of existing methods :

Existing methods either ignore the model structure and suffer from high computational cost (model-agnostic), or are fast but biased and inconsistent (heuristics), while simple path reporting fails to provide meaningful insights in the case of ensembles.

3 Key contributions

This paper introduces a new way to efficiently computes Shapley values for tree models which provides exact local explanations in polynomial time (Figure 7). This method also captures feature interaction effects. Finally, the authors show that it is possible to aggregate these local explanations to obtain global model insights.

3.1 Shapley values

Shapley values are originally derived from cooperative game theory. They provide a principled way to attribute model's prediction $f(x)$ to individual input features. In this framework, each feature can be considered as a "player" in a coalition. The goal is to fairly distribute the model's output based on each feature's marginal contribution across all possible feature subsets. This leads to the following formula that averages these contributions over all feature orderings :

$$\phi_i(f,x) = \sum_{S \subseteq \mathcal{M} \setminus \{i\}} \frac{|S|!(M-|S|-1)!}{M!} \left[f(S \cup \{i\}) - f(S)\right]$$

In practice, SHAP extends this concept to machine learning models with continuous features by relying on interventional conditional expectations :

$$f_x(S) = \mathbb{E}(f(X) \mid \text{do}(X_S = x_S))$$

This interventional conditional expectation is explained in (7). The issue with using normal conditional expectations is that they are influenced by dependencies between features in the data distribution (if there is a latent variable influencing them for example). The interventional expectation "breaks" this interdependence when computing the Shapley values, because we are only interested in the impact of the feature in the model, not in the data distribution.

SHAP values are attractive because they are the only additive feature attribution method that satisfies three key properties:

- **Local Accuracy (Efficiency)**: the prediction is exactly decomposed into the sum of feature contributions.
$$f(x) = \sum_{i=1}^{M} \phi_i(f,x)$$
- **Consistency (Monotonicity)**: if a model changes so that a feature has more impact, its attribution cannot decrease.
$$f'(S \cup \{i\}) - f'(S) \geq f(S \cup \{i\}) - f(S) \quad \Rightarrow \quad \phi_i(f',x) \geq \phi_i(f,x)$$
- **Missingness (Nullity)**: features with no effect receive zero contribution.
$$f(S \cup \{i\}) = f(S) \quad \forall S \subseteq \mathcal{M} \quad \Rightarrow \quad \phi_i(f,x) = 0$$

3.2 Bruteforce SHAP values computation on trees

The brute-force algorithm for computing SHAP values follows the original definition from Game Theory. For each feature i, it evaluates the model under all possible subsets of features $S \subseteq \mathcal{M} \setminus \{i\}$, and measures the marginal contribution of i by computing the difference in expected outputs:

$$\phi_i(f, x) = \sum_{S \subseteq \mathcal{M} \setminus \{i\}} \frac{|S|!(M - |S| - 1)!}{M!} \left(f'_x(S \cup \{i\}) - f'_x(S)\right)$$

f'_x is an approximation of $f_x(S) = \mathbb{E}(f(X) \mid \text{do}(X_S = x_S))$ computed using the training dataset. The TreeSHAP values are exact Shapley values, for this approximate function as a reward function.

Each term is weighted to ensure fairness over all possible orderings. For ensemble, models, the overall Shapley value is simply the average of the Shapley values computed across each tree:

$$\phi_i^{\text{ensemble}} = \frac{1}{T} \sum_{t=1}^{T} \phi_i^t$$

When the number of features grows, this approach becomes computationally infeasible because of the exponential number of subsets to evaluate.

3.3 Tree SHAP Algorithm : Optimized version

Tree SHAP provides an exact but more efficient alternative, tailored to tree-based models. Instead of computing expectations over all feature subsets, the algorithm recursively propagates and updates subset weights using two main procedures `EXTEND`, which pushes down the proportion of subsets through the tree nodes, and `UNWIND`, which adjusts the contributions when moving back up the tree. By exploiting the tree structure, Tree SHAP reduces the complexity from $O(TLM2^M)$ to $O(TLD^2)$, where T is the number of trees, L the maximum number of leaves per tree, M the number of input features, and D the maximum depth of a tree. Thus, it makes it feasible to compute exact Shapley values (for f'_x the approximate value function) even for large ensembles, in polynomial time.

4 Comparison between shap library and brute-force algorithm coded from scratch

We implemented the brute-force Shapley value computation algorithm (Algorithm 1 of the paper) from scratch and compared it with the optimized version available in the `shap` Python library (with `feature_perturbation="path_independent"`) . Our comparison focuses on two things: computational performance (runtime) and the actual Shapley values produced by both methods. We conducted the experiment on the mortality dataset, one of the three datasets used in the original paper.

The runtime evaluation confirms the theoretical complexity rates reported in the paper: the optimized Tree SHAP algorithm from the library shows polynomial runtime while our brute-force implementation grows exponentially with the number of features and quickly becomes impractical (Figures 2 3).

We focused on two specific features (age and body mass index) to assess the numerical agreement between the two methods. We randomly sampled 10 individuals form the dataset and computed their SHAP values using both methods. We then performed a Wilcoxon test to compare the values. In all cases, we did not reject the null hypothesis, meaning that the SHAP values obtained from both methods were statistically equivalent. We obtained the same results when focusing on three features (age, body mass index, and systolic blood pressure), although the brute-force algorithm was slower to compute.

5 Results and Applications

We now identify the strengths of the paper while trying to reproduce the results of the paper.

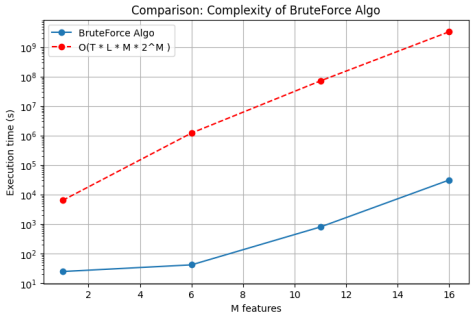

Figure 2: (a) Runtime Brute-force with respect of the number of features M (Our figure)

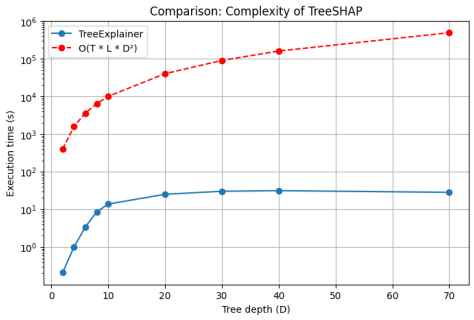

Figure 3: (b) Runtime TreeSHAP with respect of the Tree depth D (Our figure)

5.1 TreeExplainer vs Saabas: Consistency Matters

To illustrate the importance of consistency in explanation methods, the authors evaluate Saabas and Tree SHAP on synthetic k-way AND function, where all k features contribute equally to the model output. Feature attributions are computed and analyzed relatively to its depths in the tree. The results highlight that Saabas method systematically underestimates the importance of features closer to the root. This leads to inconsistent and biased explanations. In contrast, Tree SHAP gives uniform attributions across all features, which is correct because of their equal contribution. The experiment shows the advantage of Tree SHAP in terms of fairness in feature attribution.

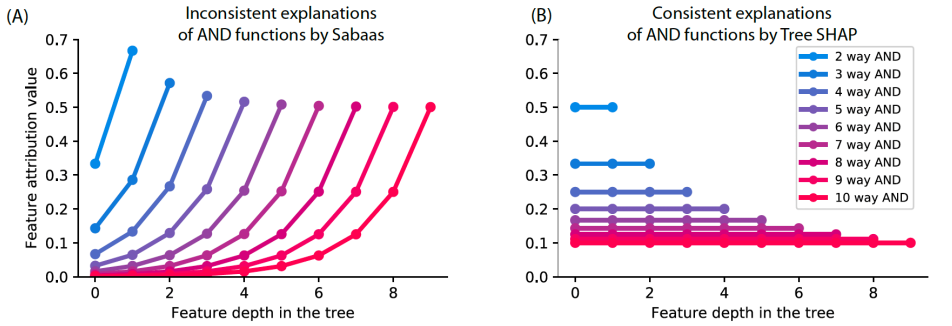

Figure 4: TreeExplainer avoids the bias problems of Saabas (taken from (1)).

Another experiment illustrates how Tree SHAP is the only method that provides local explanations with guaranteed consistency. When comparing two similar models where the importance of the feature "couch" increases, Tree SHAP reflects this change, while Saabas and Gain (global method) methods assign inconsistent or even lower attributions. This violates the consistency principle defined above.

5.2 From Local Explanations to Global Understanding

SHAP values are computed locally for each individual prediction. We can get a global view of how the model behaves by aggregating them across the dataset. We reproduced this kind of analysis and compared the results with the standard Gain feature importance from XGBoost.

While the Gain method gives a general idea of which features matter most. It does not tell us how these features really affect the predictions. That's where SHAP becomes useful: the summary plot and dependence plots not only show which features are important but also how their values influence the model.

For instance, looking at the "age" feature, we clearly see that higher age increases the predicted risk of mortality (Figure 8).

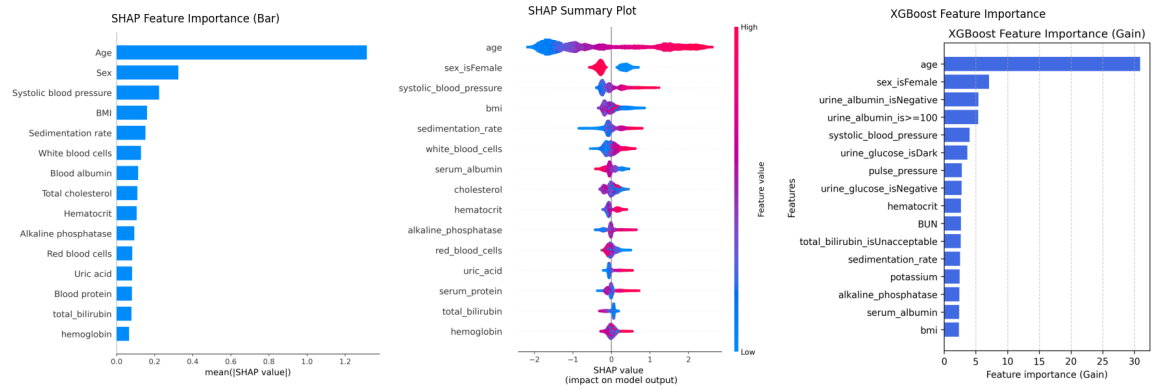

Figure 5: (Our figure) Comparison of global feature importance using SHAP and Gain. The left and center plots show SHAP-based global explanations: the bar plot (left) ranks features by mean absolute SHAP value, while the summary plot (center) shows the distribution of each feature's impact across individuals. The right plot displays classical XGBoost Gain-based feature importance.

5.3 Interaction effects

SHAP values traditionally mix main and interaction effects. It makes it hard to isolate how features interact. To adress this, the authors extend the method with SHAP interaction values which decompose each prediction into both main effect and pairwise interactions.

$$\phi_{i,j} = \frac{1}{2} \sum_{S \subseteq \mathcal{M} \setminus \{i,j\}} \frac{|S|!(M - |S| - 2)!}{(M-1)!} \cdot [f(S \cup \{i,j\}) - f(S \cup \{i\}) - [f(S \cup \{j\}) - f(S)]]$$

where $\mathcal{M}$ is the set of all Minput features, $S \subseteq \mathcal{M} \setminus \{i,j\}$, and $f(S)$ is the model output when only the features in S are known. The interaction value $\phi_{i,j}$ captures the extent to which features i and j interact in contributing to the prediction.

For instance, here we replicate the result from the paper and it reveals that the effect of systolic blood pressure on mortality risk depends strongly on age. High systolic blood pressure increases risk more in younger patients. In older individuals, the effect is weaker or even reversed (Figure 6).

So TreeExplainer does not just tell us what matters, but also for whom and under which conditions which is very useful for personalized interpretation.

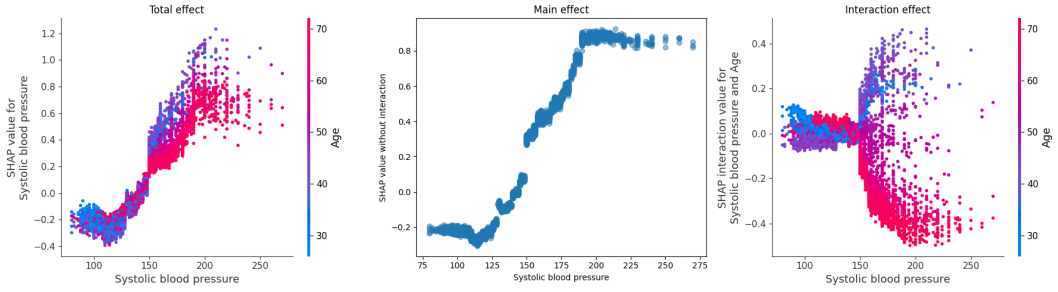

Figure 6: (Our figure) Decomposition of SHAP values for systolic blood pressure. Left: Total SHAP effect capturing both main and interaction contributions colored by age. Center: Main effect of systolic blood pressure without accounting for interactions. Right: Interaction effect between systolic blood pressure and age.

5.4 Benchmarking with Existing Explanation Methods

To benchmark TreeExplainer against existing methods, the authors evaluated several explanation techniques across 15 different metrics on the Chronic disease dataset. These include local accuracy, consistency and runtime. TreeExplainer (path-dependent) consistently outperforms other methods on almost all evaluation criteria, while maintaining a low runtime. In contrast, model-agnostic methods show more variability in performance and are slower. This highlights TreeExplainers strong balance between theoretical soundness, practical accuracy, and computational efficiency.

6 Discussion

Even though TreeExplainer is already a solid and efficient method for tree-based models, theres still room for improvement. One interesting direction would be to go beyond pairwise interactions and explore higher-order effects involving three or more features. It could also be useful to further optimize the algorithm to make it even faster and more scalable, especially when dealing with large ensembles or real-time applications as done in (2).

7 Conclusion

To sum up, the paper introduces an efficient method to compute exact SHAP values for tree-based models in polynomial time. It also extends the framework to capture interaction effects between features, providing deeper insights into model behavior. By aggregating local explanations, it enables global model understanding. It also outperforms existing methods in terms of accuracy, consistency and speed. TreeExplainer stands out as a strong and practical contribution to the field of interpretable machine learning.

References

[1] Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. *From local explanations to global understanding with explainable AI for trees*. Nature Machine Intelligence, 2(1):5667, 2020.

[2] Jilei Yang. *Fast TreeSHAP: Accelerating SHAP Value Computation for Trees*. arXiv preprint arXiv:2107.03170, 2021.

[3] Molnar, C. (2025). Interpretable Machine Learning: A Guide for Making Black Box Models Explainable (3rd ed.). christophm.github.io/interpretable-ml-book/

[4] Kuba et al., (2019). pyCeterisParibus: explaining Machine Learning models with Ceteris Paribus Profiles in Python. *Journal of Open Source Software, 4(37), 1389, https://doi.org/10.21105/joss.01389*

[5] Goldstein, A., Kapelner, A., Bleich, J., & Pitkin, E. (2015). Peeking Inside the Black Box: Visualizing Statistical Learning With Plots of Individual Conditional Expectation. *Journal of Computational and Graphical Statistics, 24(1), 4465. https://doi.org/10.1080/10618600.2014.907095*

[6] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why Should I Trust You?": Explaining the Predictions of Any Classifier. *In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16). Association for Computing Machinery, New York, NY, USA, 11351144. https://doi.org/10.1145/2939672.2939778*

[7] Janzing, D., Minorics, L. & Bloebaum, P.. (2020). Feature relevance quantification in explainable AI: A causal problem. *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics, in Proceedings of Machine Learning Research 108:2907-2916 Available from https://proceedings.mlr.press/v108/janzing20a.html*.

8 Appendix

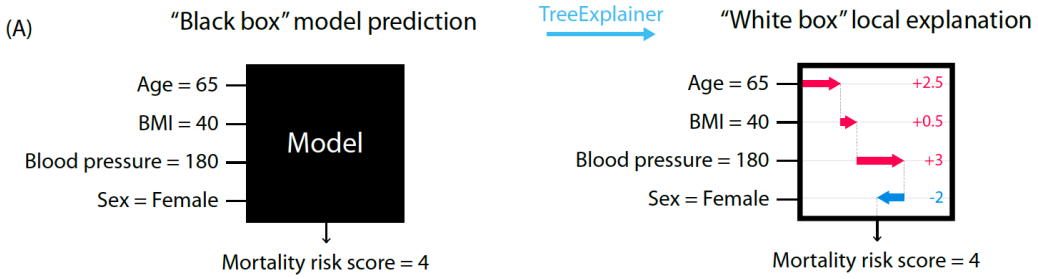

Figure 7: The objective of TreeExplainer: A local explanation based on assigning a numeric measure of credit to each input feature. (taken from (1))

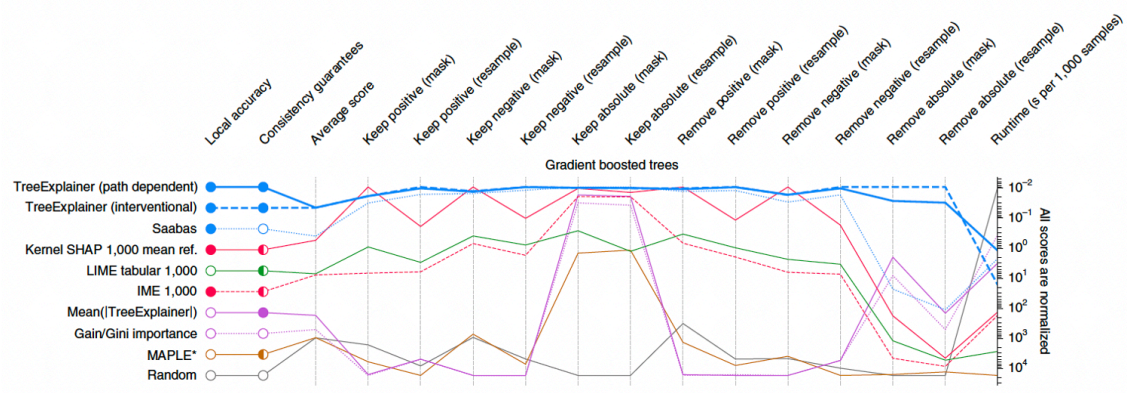

Figure 8: Figure 3 of the paper, Explanation method performance across 15 different evaluation metrics in the Chronic disease dataset. (taken from (1))